Projet AEGIS

Sécurisation d'Infrastructure

Déploiement, audit et durcissement d'un serveur Ubuntu Server 24.04 LTS

PME fictive : **TechSud** | IPSSI Lille — BTC 1

Imad ATOUANI *Responsable Sécurité*

Bilal CHAYBOUTI *Auditeur*

Mathis LETERME *Administrateur Système*

Ilyes CHOUITER *Administrateur Système*

Avril 2026

Table des matières

TP Principal – Projet AEGIS 4

Introduction générale 4

1 Contexte et audit initial de TechSud 5
- 1.1 Présentation du contexte 5
- 1.2 Analyse de la surface d'attaque 5

2 Déploiement de la VM Linux 7
- 2.1 Installation et configuration du système 7
- 2.2 Configuration réseau et accès SSH initial 8
- 2.3 Mise en place du dépôt GitHub 8

3 Durcissement du système 10
- 3.1 Sécurisation de SSH 10
- 3.2 Configuration du pare-feu ufw 12
- 3.3 Gestion des utilisateurs et des droits 12

4 Surveillance et fail2ban 16
- 4.1 Installation et configuration de fail2ban 16
- 4.2 Consultation des journaux système 17

5 Script d'audit Python 18
- 5.1 Collection des informations 18
- 5.2 Vérification de la conformité 20

6 Conformité RGPD et recommandations 25

TP Complémentaire 29

Introduction du TP complémentaire 29

7 Phase 1 : Base de données MariaDB 30
- 7.1 Installation et sécurisation initiale 30

7.2 Création d'un utilisateur restreint 32
7.3 Préparation du client Python MariaDB 33
7.4 Exposition réseau et scans Nmap 34
7.5 Injection SQL dans un script Python 35
7.6 Audit MariaDB intégré au script Python 38

8 Phase 2 : Application web et attaques 39
8.1 Installation du serveur web Apache et de PHP 39
8.2 Upload vulnérable et webshell 40
8.3 Correction de la faille d'upload 41
8.4 En-têtes HTTP et réduction de l'empreinte du serveur 42
8.5 XSS réfléchie et correction 44

9 Phase 3 : Logs et automatisation 46
9.1 Lecture manuelle des journaux 46
9.2 Script Python : analyse de auth.log 49
9.3 Script Python : analyse de Apache access.log 50
9.4 Script Bash d'audit et exécution automatique par cron 51

10 Cryptographie appliquée à TechSud 53
1. Pourquoi une clé SSH ed25519 vaut mieux qu'un mot de passe ? 53
2. Le handshake SSH, étape par étape 53
3. HTTP versus HTTPS – ce qu'un attaquant capture avec Wireshark . . . 54
4. Certificat auto-signé versus certificat signé par une CA 54

Conclusion générale 55

TP Principal – Projet AEGIS

Introduction générale

Ce compte rendu couvre l'intégralité du projet AEGIS mené dans le cadre de l'UE Sécurité des Systèmes d'Information BTC 1. Il se divise en deux parties : le TP principal, consacré au déploiement d'une infrastructure Linux sécurisée et à la production d'un rapport d'audit, et le TP complémentaire, qui approfondit la sécurisation d'une base de données MariaDB, d'une application web, et met en place une supervision automatisée des journaux.

Chapitre 1

Contexte et audit initial de TechSud

1.1 Présentation du contexte

TechSud est une PME fictive dont le système d'information a subi une compromission. L'attaquant a exploité un formulaire d'upload sans validation pour déposer un webshell PHP et obtenir une exécution de commandes sur le serveur web. Le port 3306 (MariaDB) était potentiellement exposé, SSH était accessible en `root` avec mot de passe, et aucun pare-feu n'était actif. La mission consiste à auditer, sécuriser et documenter cette infrastructure, puis à automatiser la supervision.

1.2 Analyse de la surface d'attaque

Avant toute modification, un état des lieux approfondi a été dressé pour identifier les services en écoute, les ports ouverts et les comptes présents sur la machine, ce qui a permis de mettre en évidence plusieurs vecteurs d'attaque actifs ou potentiels.

— **Webshell via formulaire de contact** (upload sans validation MIME) :
`/var/www/html/upload/shell.php` uploadé sur `SRV-WEB-01` — porte d'entrée probable de l'attaquant.

— **Compte "deploy" réactivé / mot de passe faible** :
Connexion SSH depuis `185.220.101.47` (nœud de sortie Tor) — compte normalement désactivé.

— **SSH exposé sur Internet sans restriction** :
Ports 22 ouverts sur `SRV-PROD-01` et `SRV-WEB-01`.

— **MariaDB potentiellement exposé sur WAN** :
Port 3306 accessible depuis l'extérieur.

— **Site en HTTP sans HTTPS** :
Interception possible des communications.

— **Pas de logs centralisés** :
Effacement facilité (fichier `auth.log` partiellement effacé).

— **Cron malveillant** :
`/etc/cron.d/sysupdate` exécute `/tmp/.x11-unix/sshd_bak` toutes les 5 minutes.

— **Connexion C2 vers** `45.142.212.100:4444` :
Probablement un reverse shell ou un mineur de cryptomonnaie.

Chapitre 2

Déploiement de la VM Linux

2.1 Installation et configuration du système

Une machine virtuelle sous Ubuntu Server 24.04 LTS a été déployée. La configuration réseau en mode pont (*bridged*) permet à la VM d'être accessible depuis la machine attaquante tout en restant dans un segment réseau contrôlé.

```
groupe-imad@ubuntu-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b2:c4:a7 brd ff:ff:ff:ff:ff:ff
    inet 10.74.2.69/22 metric 100 brd 10.74.3.255 scope global dynamic enp0s3
       valid_lft 7029sec preferred_lft 7029sec
    inet6 fe80::a00:27ff:feb2:c4a7/64 scope link
       valid_lft forever preferred_lft forever
groupe-imad@ubuntu-server:~$ ip route show
default via 10.74.0.1 dev enp0s3 proto dhcp src 10.74.2.69 metric 100
8.8.8.8 via 10.74.0.1 dev enp0s3 proto dhcp src 10.74.2.69 metric 100
10.74.0.0/22 dev enp0s3 proto kernel scope link src 10.74.2.69 metric 100
10.74.0.1 dev enp0s3 proto dhcp scope link src 10.74.2.69 metric 100
groupe-imad@ubuntu-server:~$ ping -c 3 google.com
PING google.com (142.251.39.110) 56(84) bytes of data.
64 bytes from pnpara-ag-in-f14.1e100.net (142.251.39.110): icmp_seq=1 ttl=113 time=12.6 ms
64 bytes from pnpara-ag-in-f14.1e100.net (142.251.39.110): icmp_seq=2 ttl=113 time=9.51 ms
64 bytes from pnpara-ag-in-f14.1e100.net (142.251.39.110): icmp_seq=3 ttl=113 time=9.79 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2009ms
rtt min/avg/max/mdev = 9.512/10.639/12.619/1.404 ms
groupe-imad@ubuntu-server:~$ hostname
ubuntu-server
groupe-imad@ubuntu-server:~$ hostnamectl
 Static hostname: ubuntu-server
       Icon name: computer-vm
         Chassis: vm 🖴
      Machine ID: 00aa00a27b504af78ced492c7f7d209b
         Boot ID: dd60e66fe72f4339b6cf04cde0c1ed60
  Virtualization: oracle
Operating System: Ubuntu 24.04.4 LTS
          Kernel: Linux 6.8.0-107-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
```

Figure 2.1 – Configuration réseau de la VM Ubuntu Server 24.04 LTS (`ip a`, `ip route show`, `hostnamectl`) – IP 10.74.2.69, kernel 6.8.0

```
groupe-imad@ubuntu-server:~$ sudo apt update && sudo apt upgrade -y
Atteint :1 http://fr.archive.ubuntu.com/ubuntu noble InRelease
Réception de :2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Atteint :3 http://fr.archive.ubuntu.com/ubuntu noble-updates InRelease
Atteint :4 http://fr.archive.ubuntu.com/ubuntu noble-backports InRelease
Réception de :5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21,5 kB]
Réception de :6 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Réception de :7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74,2 kB]
Réception de :8 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
222 ko réceptionnés en 0s (490 ko/s)
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Tous les paquets sont à jour.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Calcul de la mise à jour... Fait
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
groupe-imad@ubuntu-server:~$ _
```

Figure 2.2 – Mise à jour du système

```
groupe-imad@ubuntu-server:~$ sudo apt install -y openssh-server ufw fail2ban nmap net-tools git python3 python3-pip curl vim htop tree jq_
```

Figure 2.3 – Installation d'outils nécessaires pour la sécurisation

2.2 Configuration réseau et accès SSH initial

Après l'installation, le statut du service SSH a été vérifié avant tout durcissement afin d'établir l'état de référence.

```
groupe-imad@ubuntu-server:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-04-13 12:43:51 UTC; 36min ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 828 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 841 (sshd)
      Tasks: 1 (limit: 4601)
     Memory: 4.0M (peak: 5.0M)
        CPU: 95ms
     CGroup: /system.slice/ssh.service
             └─841 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

avril 13 12:43:50 ubuntu-server systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
avril 13 12:43:51 ubuntu-server sshd[841]: Server listening on 0.0.0.0 port 22.
avril 13 12:43:51 ubuntu-server sshd[841]: Server listening on :: port 22.
avril 13 12:43:51 ubuntu-server systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
avril 13 12:55:10 ubuntu-server sshd[1164]: Accepted password for groupe-imad from 10.74.2.22 port 61289 ssh2
avril 13 12:55:10 ubuntu-server sshd[1164]: pam_unix(sshd:session): session opened for user groupe-imad(uid=1000) by groupe-imad(uid=0)
avril 13 12:59:33 ubuntu-server sshd[1256]: Accepted password for groupe-imad from 10.74.2.22 port 59078 ssh2
avril 13 12:59:33 ubuntu-server sshd[1256]: pam_unix(sshd:session): session opened for user groupe-imad(uid=1000) by groupe-imad(uid=0)
groupe-imad@ubuntu-server:~$ sudo cat /etc/ssh/sshd_config | grep -v "^#" | grep -v "^$"
Include /etc/ssh/sshd_config.d/*.conf
KbdInteractiveAuthentication no
UsePAM yes
X11Forwarding yes
PrintMotd no
AcceptEnv LANG LC_*
Subsystem       sftp    /usr/lib/openssh/sftp-server
groupe-imad@ubuntu-server:~$
```

Figure 2.4 – Statut du service SSH avant durcissement et directives actives dans `sshd_config` par défaut – authentification par mot de passe active, port 22

2.3 Mise en place du dépôt GitHub

Un dépôt GitHub a été initialisé pour centraliser l'ensemble des livrables : scripts, fichiers de configuration et rapport. Des commits réguliers ont été effectués tout au long de la semaine pour tracer l'historique des modifications.

```
groupe-imad@ubuntu-server:~$ git config --global user.name "Imad Atouani"
groupe-imad@ubuntu-server:~$ git config --global user.email "imadatouani270@gmail.com"
groupe-imad@ubuntu-server:~$ mkdir ~/aegies-groupe-imad
groupe-imad@ubuntu-server:~$ cd ~/aegies-groupe-imad/
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/groupe-imad/aegies-groupe-imad/.git/
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ cat > README.md << 'EOF'
> # AEGIS - Groupe-Imad
> Projet sécurisation SI - BTC1 IPSSI
> Client : TechSud
>
> ## Membres
> - Imad ATOUANI
> - Bilal CHAYBOUTI
> - Mathis LETERME
> - Ilyes CHOUITER
>
> ## Structure
> - '/scripts/' - Script Python d'audit
> - '/configs/' - Fichiers de configuration
> - '/logs-captures/' - Captures des logs
> EOF
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ mkdir -p scripts configs logs-captures
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ git add .
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ git commit -m "init: structure du projet AEGIS"
[master (root-commit) 093631a] init: structure du projet AEGIS
 1 file changed, 14 insertions(+)
 create mode 100644 README.md
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ _
```

```
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ git remote set-url origin https://ImadAtouani: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@github.com/ImadAtouani/aegis-grou
pe-imad.git
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 444 bytes | 444.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ImadAtouani/aegis-groupe-imad.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ git config credential.helper store
groupe-imad@ubuntu-server:~/aegies-groupe-imad$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
groupe-imad@ubuntu-server:~/aegies-groupe-imad$
```

Figure 2.5 – Premier push du projet AEGIS vers le dépôt GitHub distant (`git push -u origin main`)

Chapitre 3

Durcissement du système

3.1 Sécurisation de SSH

SSH étant le principal vecteur d'accès à la VM, plusieurs mesures ont été appliquées successivement :

1. génération d'une paire de clés ed25519 sur la VM et ajout dans `authorized_keys` ;
2. sauvegarde de la configuration SSH existante (`sshd_config.bak`) ;
3. déplacement du service sur un port non standard (2222) ;
4. désactivation de l'authentification par mot de passe (`PasswordAuthentication no`) ;
5. interdiction de la connexion directe en tant que `root` (`PermitRootLogin no`).

```
ssh-keygen -t ed25519 -C "aegis-projet" -f ~/.ssh/aegis_key
cat ~/.ssh/aegis_key.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

Listing 3.1 – Génération de la paire de clés ed25519 et déploiement dans authorized_keys

```
adminaegis@ubuntu-server:~$ mkdir -p ~/.ssh
adminaegis@ubuntu-server:~$ chmod 700 ~/.ssh
adminaegis@ubuntu-server:~$ ssh-keygen -t ed25519 -C "aegis-projet" -f ~/.ssh/aegis_key
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/adminaegis/.ssh/aegis_key
Your public key has been saved in /home/adminaegis/.ssh/aegis_key.pub
The key fingerprint is:
SHA256:fQOc4DLgEtqBJduJXU9qU7Cm12ceNNdZ3lLlkpN4AJw aegis-projet
The key's randomart image is:
+--[ED25519 256]--+
|.o+ +.o ...o. . +|
|.O * B . oEo * * |
|+ * B + + = + O o|
|   = o + + . . + |
|  . . . S . o    |
|   .   + . . .   |
|        .        |
|                 |
|                 |
+----[SHA256]-----+
adminaegis@ubuntu-server:~$ cat ~/.ssh/aegis_key.pub  >> ~/.ssh/authorized_keys
adminaegis@ubuntu-server:~$ chmod 600 ~/.ssh/authorized_keys
adminaegis@ubuntu-server:~$ _
```

Figure 3.1 – Génération de la paire de clés ed25519 (`adminaegis`) et ajout de la clé publique dans `authorized_keys`

```
Port 2222
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowTcpForwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
```

Listing 3.2 – Directives dans `/etc/ssh/sshd_config` après durcissement

```
groupe-imad@ubuntu-server:~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
[sudo] password for groupe-imad:
groupe-imad@ubuntu-server:~$ sudo vim /etc/ssh/sshd_config
```

Figure 3.2 – Sauvegarde de `sshd_config` vers `sshd_config.bak` et ouverture en édition avec `vim`

```
groupe-imad@ubuntu-server:~$ sudo sshd -t
groupe-imad@ubuntu-server:~$ sudo systemctl reload ssh
groupe-imad@ubuntu-server:~$ sudo ss -tlnp | grep ssh
LISTEN 0      128          0.0.0.0:2222      0.0.0.0:*    users:(("sshd",pid=2392,fd=3))
LISTEN 0      128             [::]:2222         [::]:*    users:(("sshd",pid=2392,fd=4))
groupe-imad@ubuntu-server:~$
```

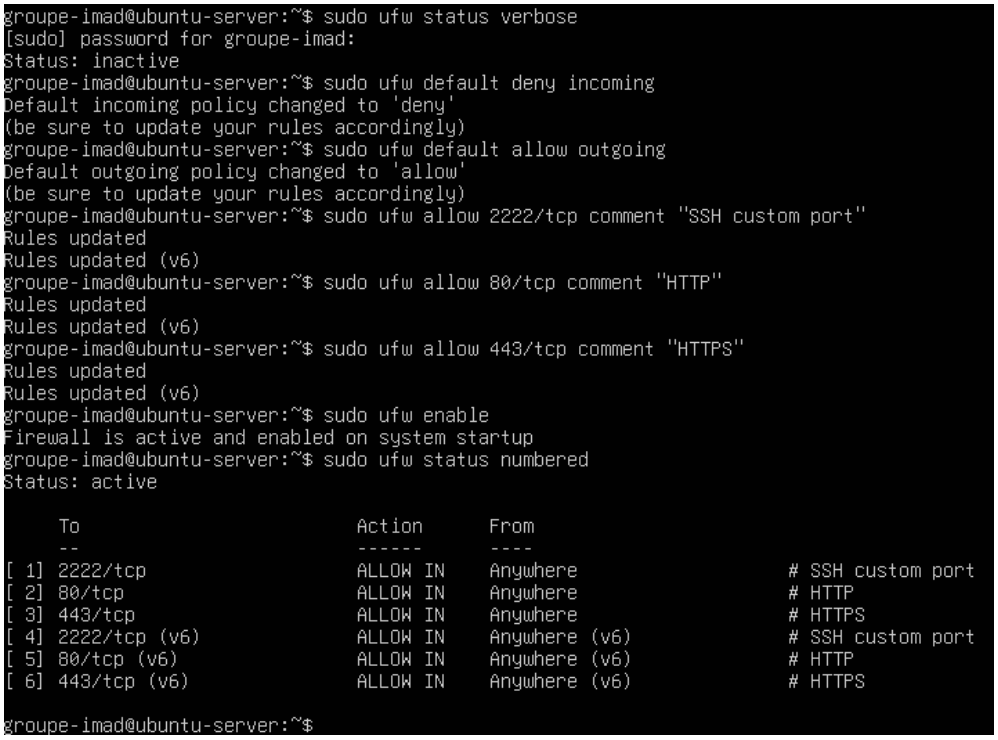
Figure 3.3 – Test de la configuration (`sshd -t`), rechargement du service et vérification de l'écoute sur le port 2222 via `ss -tlnp`

3.2 Configuration du pare-feu ufw

`ufw` a été activé avec une politique de refus par défaut pour le trafic entrant. Seuls les ports correspondant à un besoin explicitement identifié ont été autorisés.

```
sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 2222/tcp comment "SSH custom port" # SSH sur port custom
sudo ufw allow 80/tcp comment "HTTP" # HTTP Apache
sudo ufw allow 443/tcp comment "HTTPS" # HTTPS
sudo ufw enable
sudo ufw status numbered
```

Listing 3.3 – Activation et configuration du pare-feu ufw



```
groupe-imad@ubuntu-server:~$ sudo ufw status verbose
[sudo] password for groupe-imad:
Status: inactive
groupe-imad@ubuntu-server:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
groupe-imad@ubuntu-server:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
groupe-imad@ubuntu-server:~$ sudo ufw allow 2222/tcp comment "SSH custom port"
Rules updated
Rules updated (v6)
groupe-imad@ubuntu-server:~$ sudo ufw allow 80/tcp comment "HTTP"
Rules updated
Rules updated (v6)
groupe-imad@ubuntu-server:~$ sudo ufw allow 443/tcp comment "HTTPS"
Rules updated
Rules updated (v6)
groupe-imad@ubuntu-server:~$ sudo ufw enable
Firewall is active and enabled on system startup
groupe-imad@ubuntu-server:~$ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 2222/tcp                   ALLOW IN    Anywhere                   # SSH custom port
[ 2] 80/tcp                     ALLOW IN    Anywhere                   # HTTP
[ 3] 443/tcp                    ALLOW IN    Anywhere                   # HTTPS
[ 4] 2222/tcp (v6)              ALLOW IN    Anywhere (v6)              # SSH custom port
[ 5] 80/tcp (v6)                ALLOW IN    Anywhere (v6)              # HTTP
[ 6] 443/tcp (v6)               ALLOW IN    Anywhere (v6)              # HTTPS

groupe-imad@ubuntu-server:~$
```

Figure 3.4 – État du pare-feu ufw après configuration – ports 2222 (SSH), 80 (HTTP) et 443 (HTTPS) autorisés

3.3 Gestion des utilisateurs et des droits

Le compte `root` n'est plus accessible directement via SSH. Un compte opérationnel dédié a été créé pour les tâches d'administration, avec des droits `sudo` limités et journalisés dans `auth.log`.

```
sudo adduser adminaegis
sudo usermod -aG sudo adminaegis
id adminaegis
```

Listing 3.4 – Création d'un compte opérationnel avec droits sudo

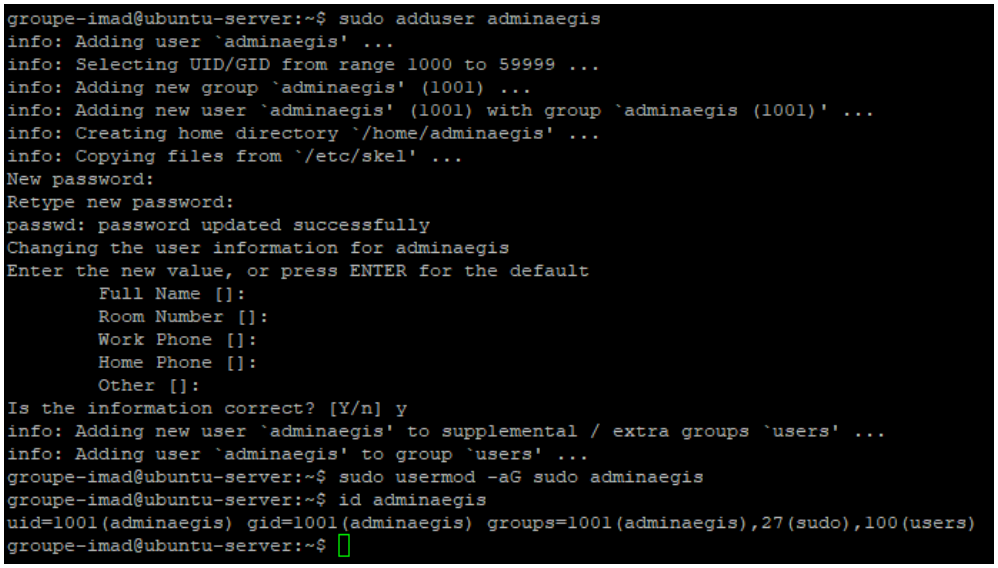


Figure 3.5 – Création du compte `adminaegis`, attribution des droits sudo et vérification de l'appartenance aux groupes

```
# Lister tous les utilisateurs avec shell actif
cat /etc/passwd | grep -v nologin | grep -v false

# D sactiver les comptes inutiles (exemple : compte "deploy" comme sur TechSud)
sudo usermod -L deploy 2>/dev/null || echo "compte deploy inexistant"
sudo usermod -s /usr/sbin/nologin deploy 2>/dev/null

# V rifier les utilisateurs avec UID 0 (root-like)
awk -F: '($3 == 0) {print}' /etc/passwd

# Lister les sudoers
sudo cat /etc/sudoers
sudo ls /etc/sudoers.d/

# V rifier les fichiers SUID suspects (vecteur d'escalade de privil ges)
sudo find / -perm -4000 -type f 2>/dev/null | sort
```

```
17
18 # V rifier les fichiers world-writable
19 sudo find /etc -writable -type f 2>/dev/null
20
21 # V rifier les permissions sur les r pertoires sensibles
22 ls -la /etc/ssh/
23 ls -la /var/log/
24 ls -la /tmp/
25
26 # Permissions saines sur les fichiers critiques
27 sudo chmod 600 /etc/ssh/sshd_config
28 sudo chmod 644 /etc/passwd
29 sudo chmod 640 /etc/shadow
30 sudo chmod 644 /etc/group
31
32 # V rifier les crontabs (simuler le cas TechSud)
33 sudo ls -la /etc/cron.d/
34 sudo cat /etc/crontab
35 sudo crontab -l 2>/dev/null
36 sudo crontab -u www-data -l 2>/dev/null
```

Listing 3.5 – Vérification des utilisateurs et des droits

```
groupe-imad@ubuntu-server:~$ cat /etc/passwd | grep -v nologin | grep -v false
root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
groupe-imad:x:1000:1000:groupe-imad:/home/groupe-imad:/bin/bash
adminaegis:x:1001:1001:,,,:/home/adminaegis:/bin/bash
groupe-imad@ubuntu-server:~$ sudo usermod -L deploy 2>/dev/null || echo "compte deploy inexistant"
compte deploy inexistant
groupe-imad@ubuntu-server:~$ sudo usermod -s /usr/sbin/nologin deploy 2>/dev/null
groupe-imad@ubuntu-server:~$ awk -F: '($3 == 0) {print}' /etc/passwd
root:x:0:0:root:/root:/bin/bash
groupe-imad@ubuntu-server:~$
```

```
groupe-imad@ubuntu-server:~$ sudo ls /etc/sudoers.d/
README
groupe-imad@ubuntu-server:~$ sudo find / -perm -4000 -type f 2>/dev/null | sort
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/fusermount3
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
/usr/bin/umount
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/mysql/plugin/auth_pam_tool_dir/auth_pam_tool
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
groupe-imad@ubuntu-server:~$ _
```

```
groupe-imad@ubuntu-server:~$ ls -la /etc/ssh/
total 664
drwxr-xr-x   4 root root   4096 avril 13 14:08 .
drwxr-xr-x 114 root root   4096 avril 13 14:42 ..
-rw-r--r--   1 root root 620042 mars   4 17:55 moduli
-rw-r--r--   1 root root   1649 août  26  2025 ssh_config
drwxr-xr-x   2 root root   4096 août  26  2025 ssh_config.d
-rw-r--r--   1 root root   3481 avril 13 14:08 sshd_config
-rw-r--r--   1 root root   3517 avril 13 13:51 sshd_config.bak
drwxr-xr-x   2 root root   4096 mars   4 17:55 sshd_config.d
-rw-------   1 root root    513 avril 13 11:35 ssh_host_ecdsa_key
-rw-r--r--   1 root root    180 avril 13 11:35 ssh_host_ecdsa_key.pub
-rw-------   1 root root    411 avril 13 11:35 ssh_host_ed25519_key
-rw-r--r--   1 root root    100 avril 13 11:35 ssh_host_ed25519_key.pub
-rw-------   1 root root   2602 avril 13 11:35 ssh_host_rsa_key
-rw-r--r--   1 root root    572 avril 13 11:35 ssh_host_rsa_key.pub
-rw-r--r--   1 root root    342 déc.   7  2020 ssh_import_id
groupe-imad@ubuntu-server:~$ _
```

```
groupe-imad@ubuntu-server:~$ ls -la /tmp/
total 52
drwxrwxrwt 13 root root 4096 avril 13 23:40 .
drwxr-xr-x 23 root root 4096 avril 13 11:31 ..
drwxrwxrwt  2 root root 4096 avril 13 23:40 .font-unix
drwxrwxrwt  2 root root 4096 avril 13 23:40 .ICE-unix
drwx------  2 root root 4096 avril 13 23:40 snap-private-tmp
drwx------  3 root root 4096 avril 13 23:40 systemd-private-bfe596cbceae425dbc4ec7a1ccc30da7-apache2.service-pJhvcs
drwx------  3 root root 4096 avril 13 23:40 systemd-private-bfe596cbceae425dbc4ec7a1ccc30da7-ModemManager.service-1D6ITx
drwx------  3 root root 4096 avril 13 23:40 systemd-private-bfe596cbceae425dbc4ec7a1ccc30da7-polkit.service-CkngOQ
drwx------  3 root root 4096 avril 13 23:40 systemd-private-bfe596cbceae425dbc4ec7a1ccc30da7-systemd-logind.service-CWT8ST
drwx------  3 root root 4096 avril 13 23:40 systemd-private-bfe596cbceae425dbc4ec7a1ccc30da7-systemd-resolved.service-PmQYSF
drwx------  3 root root 4096 avril 13 23:40 systemd-private-bfe596cbceae425dbc4ec7a1ccc30da7-systemd-timesyncd.service-uZDPT8
drwxrwxrwt  2 root root 4096 avril 13 23:40 .X11-unix
drwxrwxrwt  2 root root 4096 avril 13 23:40 .XIM-unix
groupe-imad@ubuntu-server:~$ _
```

```
groupe-imad@ubuntu-server:~$ sudo crontab -l 2>/dev/null
groupe-imad@ubuntu-server:~$ sudo crontab -u www-data -l 2>/dev/null
groupe-imad@ubuntu-server:~$ sudo systemctl list-units --type=service --state=running > ~/aegis-groupe-imad/configs/services-actifs.txt
groupe-imad@ubuntu-server:~$ sudo ufw status verbose > ~/aegis-groupe-imad/configs/ufw-rules.txt
groupe-imad@ubuntu-server:~$ sudo cp /etc/fail2ban/jail.local ~/aegis-groupe-imad/configs/fail2ban-jail.local
groupe-imad@ubuntu-server:~$ git -C ~/aegis-groupe-imad/ add .
groupe-imad@ubuntu-server:~$ git -C ~/aegis-groupe-imad/ commit -m "ufw configuré, fail2ban actif, permissions vérifiées"
[main f335d2a] ufw configuré, fail2ban actif, permissions vérifiées
 3 files changed, 1026 insertions(+)
 create mode 100644 configs/fail2ban-jail.local
 create mode 100644 configs/services-actifs.txt
 create mode 100644 configs/ufw-rules.txt
groupe-imad@ubuntu-server:~$ git -C ~/aegis-groupe-imad/ push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 9.61 KiB | 9.61 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ImadAtouani/aegis-groupe-imad.git
   5c3c3dc..f335d2a  main -> main
groupe-imad@ubuntu-server:~$ _
```

Figure 3.6 – Vérification des utilisateurs et des droits

Chapitre 4

Surveillance et fail2ban

4.1 Installation et configuration de fail2ban

`fail2ban` surveille les journaux d'authentification et bannit automatiquement les adresses IP qui dépassent un seuil d'échecs consécutifs. La configuration a été placée dans `/etc/fail2ban/jail.local` pour ne pas modifier les fichiers par défaut, susceptibles d'être écrasés lors des mises à jour.

```
[DEFAULT]
bantime  = 1h
findtime  = 10m
maxretry = 3
ignoreip = 127.0.0.1/8

[sshd]
enabled = true
port    = 2222
logpath = %(sshd_log)s
backend = %(sshd_backend)s
maxretry = 3
bantime = 24h
```

Listing 4.1 – Configuration de `jail.local` pour la surveillance SSH

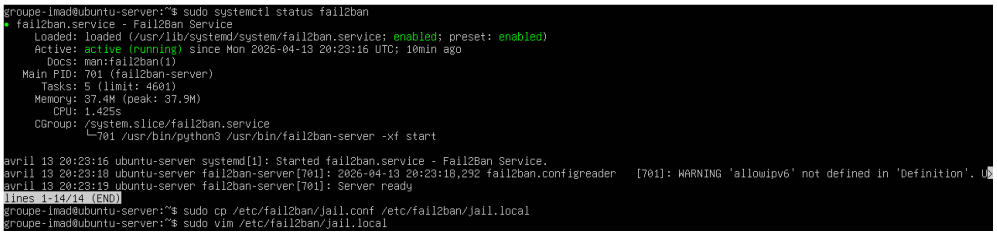


Figure 4.1 – Statut du service `fail2ban` (actif) et copie de `jail.conf` vers `jail.local` pour la configuration personnalisée

Trois connexions SSH échouées en moins de dix minutes déclenchent le bannissement automatique de l'adresse IP source, confirmé dans `/var/log/fail2ban.log`.

4.2 Consultation des journaux système

La lecture de `/var/log/auth.log` a permis de vérifier le bon fonctionnement du bannissement et de confirmer que les connexions légitimes par clé continuaient à passer.

```
groupe-imad@ubuntu-server:~$ sudo tail -10 /var/log/auth.log
2026-04-14T01:50:14.000532+00:00 ubuntu-server sudo: pam_unix(sudo:session): session closed for user root
2026-04-14T01:50:14.804820+00:00 ubuntu-server sshd[2392]: Server listening on 0.0.0.0 port 2222.
2026-04-14T01:50:14.805010+00:00 ubuntu-server sshd[2392]: Server listening on :: port 2222.
2026-04-14T01:50:17.790355+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/ss -tlnp
2026-04-14T01:50:17.791128+00:00 ubuntu-server sudo: pam_unix(sudo:session): session opened for user root(uid=0) by groupe-imad(uid=1000)
2026-04-14T01:50:17.797249+00:00 ubuntu-server sudo: pam_unix(sudo:session): session closed for user root
2026-04-14T01:55:01.881255+00:00 ubuntu-server CRON[3070]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-04-14T01:55:01.884889+00:00 ubuntu-server CRON[3070]: pam_unix(cron:session): session closed for user root
2026-04-14T01:57:48.172286+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/tail -10 /var/log/auth.log
2026-04-14T01:57:48.177160+00:00 ubuntu-server sudo: pam_unix(sudo:session): session opened for user root(uid=0) by groupe-imad(uid=1000)
groupe-imad@ubuntu-server:~$ sudo grep "Failed password" /var/log/auth.log
2026-04-14T00:18:45.455618+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep 'Failed password' /var/l
og/auth.log
2026-04-14T00:19:30.601798+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep 'Failed password' /var/l
og/auth.log
2026-04-14T01:58:07.374402+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep 'Failed password' /var/l
og/auth.log
groupe-imad@ubuntu-server:~$ sudo grep "Invalid user" /var/log/auth.log
2026-04-14T00:19:50.789662+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep 'Invalid user' /var/log/
auth.log
2026-04-14T01:58:24.549442+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep 'Invalid user' /var/log/
auth.log
groupe-imad@ubuntu-server:~$ sudo grep "Accepted" /var/log/auth.log
2026-04-13T12:55:10.785378+00:00 ubuntu-server sshd[1164]: Accepted password for groupe-imad from 10.74.2.22 port 61289 ssh2
2026-04-13T12:59:33.841599+00:00 ubuntu-server sshd[1256]: Accepted password for groupe-imad from 10.74.2.22 port 59078 ssh2
2026-04-14T00:20:07.622172+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep Accepted /var/log/auth.l
og
2026-04-14T01:39:15.726605+00:00 ubuntu-server sshd[2918]: Accepted password for groupe-imad from 192.168.1.84 port 49175 ssh2
2026-04-14T01:58:41.370243+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep Accepted /var/log/auth.l
og
groupe-imad@ubuntu-server:~$
```

Figure 4.2 – Analyse de `auth.log` : échecs SSH (`Failed password`, `Invalid user`) et connexions réussies (`Accepted password`) depuis différentes IP

Chapitre 5

Script d'audit Python

Un script Python d'audit a été développé pour automatiser l'inventaire des informations et la vérification de conformité.

5.1 Collection des informations

Le script collecte les métadonnées telles que la date, le hostname et l'IP de la cible. Il collecte aussi les informations du système comme la version, le nom du kernel et du système d'exploitation. De plus, il collecte les informations du réseau. Le script assure aussi la collection des utilisateurs, des ports et des services actifs.

```
def collect_meta():
    print("[*] Collecte des m tadonn es...")
    results["meta"] = {
        "date": datetime.datetime.now().isoformat(),
        "auditor": os.getenv("USER", "inconnu"),
        "hostname": socket.gethostname(),
        "ip_cible": SCAN_TARGET,
        "script_version": "1.1"
    }

def collect_system():
    print("[*] Collecte des infos syst me...")
    results["system"] = {
        "os": platform.system(),
        "release": platform.release(),
        "version": run("lsb_release -d | cut -d: -f2"),
        "kernel": run("uname -r"),
        "uptime": run("uptime -p"),
        "last_update": run("stat -c %y /var/cache/apt/
            pkgcache.bin 2>/dev/null | cut -d' ' -f1"),
        "pending_updates": run("apt list --upgradable 2>/dev/
            null | grep -c upgradable") or "0",
```

```
21      }
22
23  def collect_network():
24      print("[*] Collecte des infos r seau...")
25      results["network"] = {
26          "interfaces": run("ip -brief addr"),
27          "routes": run("ip route"),
28          "listening_ports": run("ss -tlnup 2>/dev/null ||
               netstat -tlnup 2>/dev/null"),
29      }
30
31  def scan_ports():
32      print(f"[*] Scan des ports sur {SCAN_TARGET}...")
33      for port in PORTS_TO_CHECK:
34          open_ = check_port(SCAN_TARGET, port)
35          risk = "CRITIQUE" if port in [21,23,3306,3389,445]
               else \
36                  "  LEV  " if port in [22,80] else "FAIBLE"
37          results["ports"].append({
38              "port": port,
39              "open": open_,
40              "risk": risk if open_ else "N/A",
41              "note": get_port_note(port) if open_ else ""
42          })
43          status = "OUVERT    " if open_ else "ferm "
44          print(f"    Port {port:5d} : {status}")
45
46  def get_port_note(port):
47      notes = {
48          21: "FTP     transfert non chiffr ",
49          22: "SSH     v rifier durcissement",
50          23: "Telnet     NON CHIFFR , d sactiver",
51          80: "HTTP     pas de HTTPS",
52          445: "SMB     risque   lev   si expos  WAN",
53          3306: "MariaDB/MySQL     NE PAS exposer sur WAN",
54          3389: "RDP     cible fr quente de bruteforce",
55          2222: "SSH port custom     OK si durci",
56          5001: "Synology DSM     acc s admin NAS",
57      }
58      return notes.get(port, "")
59
```

```python
def collect_users():
    print("[*] Analyse des utilisateurs...")
    raw = run("cat /etc/passwd")
    for line in raw.splitlines():
        parts = line.split(":")
        if len(parts) < 7:
            continue
        username, _, uid, gid, _, home, shell = parts[:7]
        uid = int(uid)
        active = shell not in ["/usr/sbin/nologin", "/bin/false", "/sbin/nologin"]
        risk = ""
        if uid == 0 and username != "root":
            risk = "CRITIQUE     UID 0 non-root"
        elif active and uid >= 1000:
            risk = "V RIFIER"
        results["users"].append({
            "username": username,
            "uid": uid,
            "shell": shell,
            "active": active,
            "risk": risk
        })

def collect_services():
    print("[*] Collecte des services actifs...")
    raw = run("systemctl list-units --type=service --state=running --no-legend")
    for line in raw.splitlines():
        parts = line.split()
        if parts:
            results["services"].append(parts[0])
```

5.2 Vérification de la conformité

Le script vérifie la bonne configuration de UFW, Fail2ban et SSH. Il permet aussi l'export des résultats dans un fichier JSON et un fichier CSV. Finalement, le script produit un résumé de conformité.

```python
def check_ssh():
    print("[*] V rification configuration SSH...")
```

```
    checks = {
        "PermitRootLogin": ("no", "CRITIQUE"),
        "PasswordAuthentication": ("no", "  LEV  "),
        "Port": ("2222", "INFO"),
        "MaxAuthTries": ("3", "MOYEN"),
        "X11Forwarding": ("no", "FAIBLE"),
    }
    config_raw = run("sudo sshd -T 2>/dev/null")
    ssh_conf = {}
    for key, (expected, severity) in checks.items():
        val = run(f"echo '{config_raw}' | grep -i '^{key.lower()} ' | awk '{{print $2}}'")
        ok = val.lower() == expected.lower()
        ssh_conf[key] = {
            "value": val or "non trouv ",
            "expected": expected,
            "ok": ok,
            "severity": severity if not ok else "OK"
        }
        if not ok:
            results["vulnerabilities"].append({
                "id": f"SSH-{key}",
                "severity": severity,
                "description": f"SSH: {key} = '{val}' (attendu: '{expected}')",
                "recommendation": f"Modifier /etc/ssh/sshd_config : {key} {expected}"
            })
    results["ssh_config"] = ssh_conf

def check_ufw():
    print("[*] V rification UFW...")
    status = run("sudo ufw status verbose 2>/dev/null")
    results["ufw_status"] = status
    if "inactive" in status.lower() or not status:
        results["vulnerabilities"].append({
            "id": "FW-001",
            "severity": "CRITIQUE",
            "description": "Pare-feu UFW inactif ou non configur ",
            "recommendation": "Activer ufw : sudo ufw enable"
```

```
        })

def check_fail2ban():
    print("[*] V rification fail2ban...")
    status = run("sudo fail2ban-client status 2>/dev/null")
    results["fail2ban_status"] = status
    if not status:
        results["vulnerabilities"].append({
            "id": "F2B-001",
            "severity": "  LEV  ",
            "description": "fail2ban non actif",
            "recommendation": "Installer et d marrer fail2ban"
        })

def export_json():
    with open(OUTPUT_JSON, "w", encoding="utf-8") as f:
        json.dump(results, f, indent=2, ensure_ascii=False)
    print(f"[+] R sultats export s : {OUTPUT_JSON}")

def export_csv():
    with open(OUTPUT_CSV, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Port", "Ouvert", "Risque", "Note"])
        for p in results["ports"]:
            writer.writerow([p["port"], p["open"], p["risk"], p["note"]])
    print(f"[+] R sultats CSV export s : {OUTPUT_CSV}")

def print_summary():
    print("\n" + "="*50)
    print("  R SUM  DE L'AUDIT")
    print("="*50)
    open_ports = [p for p in results["ports"] if p["open"]]
    print(f"  IP audit e        : {SCAN_TARGET}")
    print(f"  Ports ouverts     : {len(open_ports)}/{len(PORTS_TO_CHECK)}")
    print(f"  Vuln rabilit s    : {len(results['vulnerabilities'])}")
    critiques = [v for v in results["vulnerabilities"] if v["
```

```
        severity"] == "CRITIQUE"]
    print(f"       Critiques       : {len(critiques)}")
    print()
    if results["vulnerabilities"]:
        print("  VULN RABILIT S  D TECT ES :")
        for v in results["vulnerabilities"]:
            print(f"  [{v['severity']:8s}] {v['id']}     {v['
                description']}")
    print("="*50)

if __name__ == "__main__":
    print("\n AEGIS     Script d'audit de s curit ")
    print(f"   Cible : {SCAN_TARGET}  |  {datetime.datetime.
        now().strftime('%d/%m/%Y %H:%M')}\n")

    collect_meta()
    collect_system()
    collect_network()
    scan_ports()
    collect_users()
    collect_services()
    check_ssh()
    check_ufw()
    check_fail2ban()
    export_json()
    export_csv()
    print_summary()
```

```
groupe-imad@ubuntu-server:~$ chmod +x ~/aegis-groupe-imad/scripts/audit.py
groupe-imad@ubuntu-server:~$ cd ~/aegis-groupe-imad/scripts/
groupe-imad@ubuntu-server:~/aegis-groupe-imad/scripts$ sudo python3 audit.py

♦  AEGIS — Script d'audit de sécurité
   Cible : 127.0.0.1  |  14/04/2026 08:02

[*] Collecte des métadonnées...
[*] Collecte des infos système...
[*] Collecte des infos réseau...
[*] Scan des ports sur 127.0.0.1...
    Port    21 : fermé
    Port    22 : fermé
    Port    23 : fermé
    Port    25 : fermé
    Port    80 : OUVERT ♦
    Port   443 : fermé
    Port   445 : fermé
    Port  2222 : OUVERT ♦
    Port  3306 : OUVERT ♦
    Port  3389 : fermé
    Port  5001 : fermé
    Port  8080 : fermé
    Port  8443 : fermé
[*] Analyse des utilisateurs...
[*] Collecte des services actifs...
[*] Vérification configuration SSH...
[*] Vérification UFW...
[*] Vérification fail2ban...
[+] Résultats exportés : audit_result.json
[+] Résultats CSV exportés : audit_result.csv

==================================================
  RÉSUMÉ DE L'AUDIT
==================================================
  Ports ouverts     : 3/13
  Vulnérabilités    : 0
  ♦  Critiques       : 0

==================================================
```

Figure 5.1 – Exécution du script d'audit Python – scan des ports (80 et 2222 ouverts, 3306 local), vérifications de conformité et export `audit_result.json` / `audit_result.csv`

Chapitre 6

Conformité RGPD et recommandations

L'audit de l'infrastructure TechSud a identifié plusieurs points de non-conformité et des axes d'amélioration au regard du RGPD et des bonnes pratiques de sécurité.

— **Traçabilité des accès :** les journaux (SSH, Apache, MariaDB) doivent être conservés conformément à la politique interne et aux obligations légales, puis archivés ou supprimés. La CNIL précise que les logs, contenant des données personnelles (adresses IP, identifiants, horodatages), doivent être pertinents, limités dans le temps et sécurisés[1]. Une conservation maximale d'un an est recommandée pour les logs de connexion, avec mise en place de politiques de rétention automatisées[2]. La centralisation mise en place dans le TP complémentaire répond à cette exigence.

— **Chiffrement des données en transit :** tout service accessible depuis l'extérieur doit être protégé par TLS. L'adoption de HTTPS pour les interfaces web de TechSud est obligatoire, idéalement via un certificat Let's Encrypt, une autorité de certification gratuite et automatisée qui fournit des certificats TLS[3]. Let's Encrypt est un projet de l'Internet Security Research Group (ISRG), organisation à but non lucratif dédiée à la sécurisation d'Internet[4].

— **Principe du moindre privilège :** aucun compte applicatif ne doit disposer de droits d'administration. Ce principe de cybersécurité, largement considéré comme une bonne pratique fondamentale, consiste à accorder à chaque utilisateur, application ou système le niveau d'accès minimum requis pour accomplir ses tâches[5]. La mise en œuvre du moindre privilège réduit la surface d'attaque, stoppe la propagation des logiciels malveillants et facilite la conformité lors des audits[6]. Les comptes de base de données (`appuser`) et les comptes système ont été restreints en conséquence.

— **Gestion des authentifiants :** l'authentification par mot de passe SSH a été remplacée par des clés ed25519. Les clés SSH offrent une sécurité renforcée par rapport aux mots de passe, étant pratiquement impossibles à déchiffrer par attaques par force brute, et éliminent les risques d'interception sur le réseau[7]. La clé privée, qui doit rester strictement confidentielle, est protégée par une

phrase de passe (passphrase) qui ajoute une couche de sécurité supplémentaire[8]. Pour les services restants, une politique de complexité et de rotation régulière est recommandée. Les credentials ne doivent jamais apparaître en clair dans le code source.

— **Mises à jour de sécurité :** le script Bash d'audit (phase 3 du TP complémentaire) vérifie automatiquement la disponibilité de correctifs. Il est recommandé d'activer `unattended-upgrades` pour les paquets de sécurité critiques. Cette fonctionnalité d'Ubuntu garantit que les correctifs de sécurité importants sont automatiquement téléchargés et installés sans intervention manuelle[9]. Canonical, l'éditeur d'Ubuntu, préinstalle et active ce package par défaut, soulignant son importance pour la sécurité des systèmes[10].

— **Réduction de la surface d'exposition :** tous les ports non nécessaires restent fermés. Le port 3306 de MariaDB doit rester joignable uniquement en local (`bind-address = 127.0.0.1`). Cette configuration, documentée par MariaDB, empêche les connexions distantes directes à la base de données et force les applications à passer par des couches intermédiaires sécurisées[11].

Bibliographie

[1] CNIL. *Logs et données de connexion : les règles à respecter*. https://www.cnil.fr/fr/logs-et-donnees-de-connexion-les-regles-respecter

[2] Donnees.net. *Gestion des logs : restez conforme au RGPD*. https://www.donnees.net/guides/rgpd/gestion-des-logs

[3] Let's Encrypt. *Encryption for Everybody*. https://letsencrypt.org/fr/

[4] ISRG. *Internet Security Research Group*. https://www.abetterinternet.org/

[5] CyberArk. *Qu'est-ce que le moindre privilège ? - Définition*. https://www.cyberark.com/fr/resources/what-is-least-privilege/

[6] IS Decisions. *Le principe du moindre privilège : la gestion de toutes les connexions d'utilisateurs*. https://www.isdecisions.com/fr/principe-moindre-privilege/

[7] DataCamp. *Les clés SSH expliquées : Guide pour un accès à distance rapide et sécurisé*. https://www.datacamp.com/fr/blog/ssh-keys-guide

[8] NixSoftware. *Sécurisez vos connexions SSH : guide complet sur l'authentification SSH par clés*. https://nixsoftware.com/blog/ssh-keys-authentication-guide/

[9] Kolide. *How to Ensure Ubuntu's Unattended Upgrades Are Turned On*. https://www.kolide.com/blog/how-to-ensure-ubuntu-s-unattended-upgrades-are-turned-on

[10] Canonical. *Security features in Ubuntu*. https://ubuntu.com/security/features

[11] MariaDB. *Remote Client Access - Server Configuration*. https://mariadb.com/kb/en/configuring-mariadb-for-remote-client-access/

TP Complémentaire

Introduction du TP complémentaire

Cette partie approfondit trois domaines spécifiques identifiés dans le dossier Tech-Sud : la base de données MariaDB (port 3306 potentiellement exposé), l'application web (formulaire d'upload sans validation ayant servi au dépôt du webshell), et la supervision des journaux (absence totale de centralisation).

Chapitre 7

Phase 1 : Base de données MariaDB

7.1 Installation et sécurisation initiale

MariaDB a été installée avec la commande `apt install mariadb-server`. La première étape de durcissement a consisté à exécuter `mysql_secure_installation` afin de supprimer les options dangereuses laissées actives par défaut.

```
groupe-imad@ubuntu-server:~$ sudo mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
      SERVERS IN PRODUCTION USE!  PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] n
 ... skipping.

You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n] n
 ... skipping.

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them.  This is intended only for testing, and to make the installation
go a bit smoother.  You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] y
 ... Success!

Normally, root should only be allowed to connect from 'localhost'.  This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.
```

```
By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them.  This is intended only for testing, and to make the installation
go a bit smoother.  You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] y
 ... Success!

Normally, root should only be allowed to connect from 'localhost'.  This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] y
 ... Success!

Cleaning up...

All done!  If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
groupe-imad@ubuntu-server:~$
```

Figure 7.1 – Exécution du script `mysql_secure_installation` – réponses aux questions de sécurité (suppression comptes anonymes, interdiction root distant, suppression base test)

Détail des choix effectués et justifications :

— **Supprimer les comptes anonymes :** accepté. Les comptes anonymes permettent à n'importe qui de se connecter à la base sans s'authentifier. C'est une surface d'attaque évidente qu'il n'y a aucune raison de conserver en production.

— **Interdire la connexion `root` à distance :** accepté. Le compte `root` SQL ne doit jamais être joignable depuis le réseau. Toute administration légitime passe par une connexion locale ou SSH, ce qui restreint considérablement l'exposition.

— **Supprimer la base `test` :** accepté. Cette base, accessible par défaut à tous les utilisateurs, n'a aucune utilité en production et est fréquemment ciblée par des scripts d'attaque automatisés.

— **Recharger les tables de privilèges :** accepté. Cette étape applique immédiatement toutes les modifications précédentes sans nécessiter un redémarrage du service.

7.2 Création d'un utilisateur restreint

Un compte applicatif dédié a été créé afin d'éviter l'utilisation du compte administrateur pour les accès courants à la base.

```sql
CREATE DATABASE techsud_db;
GRANT SELECT, INSERT, UPDATE ON techsud_db.*
    TO 'appuser'@'localhost' IDENTIFIED BY 'AppUser@2026!';
FLUSH PRIVILEGES;
```

Listing 7.1 – Création de la base et de l'utilisateur applicatif

Le principe du moindre privilège est respecté : l'utilisateur `appuser` peut lire et modifier les données nécessaires, mais il n'a pas les droits `DROP`, `DELETE` ni les droits d'administration sur la structure. Les erreurs retournées lors des tests illustrent concrètement ces restrictions :

```
ERROR 1142 (42000): DROP command denied to user
    'appuser'@'localhost' for table 'clients'
```

Listing 7.2 – Erreur retournée lors d'une tentative de `DROP TABLE` avec `appuser`

```
ERROR 1130 (HY000): Host '192.168.1.100' is not allowed
    to connect to this MariaDB server
```

Listing 7.3 – Erreur retournée lors d'une connexion distante depuis la VM attaquante

```
groupe-imad@ubuntu-server:~$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 38
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE techsud_db;
Query OK, 1 row affected (0,000 sec)

MariaDB [(none)]> CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'AppUser@2026!
';
Query OK, 0 rows affected (0,012 sec)

MariaDB [(none)]> GRANT SELECT, INSERT, UPDATE ON techsud_db.* TO 'appuser'@'loc
alhost';
Query OK, 0 rows affected (0,011 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,001 sec)

MariaDB [(none)]> EXIT;
Bye
groupe-imad@ubuntu-server:~$ mysql -u appuser -p techsud_db
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 39
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [techsud_db]> DROP TABLE clients;
ERROR 1142 (42000): DROP command denied to user 'appuser'@'localhost' for table `t
echsud_db`.`clients`
MariaDB [techsud_db]> EXIT
Bye
groupe-imad@ubuntu-server:~$
```

Figure 7.2 – Création de la base `techsud_db`, de l'utilisateur `appuser` avec `GRANT SELECT, INSERT, UPDATE`, puis vérification : tentative `DROP TABLE` refusée avec `ERROR 1142`

7.3 Préparation du client Python MariaDB

Afin d'automatiser les tests et les audits, un utilisateur `audituser` en lecture seule a d'abord été créé et les variables sensibles isolées dans un fichier `.env` exclu du dépôt Git. Le connecteur Python de MariaDB a ensuite été installé.

```
groupe-imad@ubuntu-server:~$ sudo mysql -u root -p << 'SQL'
CREATE USER IF NOT EXISTS 'audituser'@'localhost' IDENTIFIED BY 'Audit@ReadOnly202
6!';
GRANT SELECT ON mysql.* TO 'audituser'@'localhost';
FLUSH PRIVILEGES;
SQL
Enter password:
groupe-imad@ubuntu-server:~$ mkdir ~/aegis-groupe-imad/tp_complementaire
groupe-imad@ubuntu-server:~$ mkdir ~/aegis-groupe-imad/tp_complementaire/phase1
groupe-imad@ubuntu-server:~$ mkdir ~/aegis-groupe-imad/tp_complementaire/phase2
groupe-imad@ubuntu-server:~$ mkdir ~/aegis-groupe-imad/tp_complementaire/phase3
groupe-imad@ubuntu-server:~$ cd ~/aegis-groupe-imad/tp_complementaire/phase1
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase1$ cat > .env
 << 'EOF'
DB_AUDIT_USER=audituser
DB_AUDIT_PASS=Audit@ReadOnly2026!
EOF
```

Figure 7.3 – Création de l'utilisateur `audituser` (`GRANT SELECT` sur `mysql.*`), préparation des répertoires `tp_complementaire/` et fichier `.env` avec les credentials

```
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase1$ echo ".env" >> ~/aegis-groupe-imad/.gitignore
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase1$ cd ~
groupe-imad@ubuntu-server:~$ pip3 install mysql-connector-python --break-system-pa
ckages
Defaulting to user installation because normal site-packages is not writeable
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.6.0-cp312-cp312-manylinux_2_28_x86_64.whl.m
etadata (11 kB)
Downloading mysql_connector_python-9.6.0-cp312-cp312-manylinux_2_28_x86_64.whl (34
.5 MB)
   ---------------------------------------- 34.5/34.5 MB 1.2 MB/s eta 0:00:00
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.6.0
groupe-imad@ubuntu-server:~$
```

Figure 7.4 – Ajout de `.env` dans `.gitignore` et installation du connecteur `mysql-connector-python` via `pip3`

7.4 Exposition réseau et scans Nmap

Par défaut, MariaDB n'écoute que sur `127.0.0.1`. Pour démontrer l'impact d'une mauvaise exposition, la directive `bind-address` du fichier `/etc/mysql/mariadb.conf.d/50-server.cn` a été temporairement modifiée en `0.0.0.0`, puis rétablie après vérification.

```
groupe-imad@ubuntu-server:~$ sudo mysql -u root -p techsud_db << 'SQL'
CREATE TABLE IF NOT EXISTS clients (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    nom   VARCHAR(100),
    email VARCHAR(100)
);
INSERT INTO clients (nom, email) VALUES
    ('Dupont',  'dupont@techsud.fr'),
    ('Martin',  'martin@techsud.fr'),
    ('Bernard', 'bernard@techsud.fr');
SQL
Enter password:
groupe-imad@ubuntu-server:~$ sudo sed -i 's/^bind-address.*/bind-address = 0.0.0.0
/' /etc/mysql/mariadb.conf.d/50-server.cnf
groupe-imad@ubuntu-server:~$ sudo systemctl restart mariadb
groupe-imad@ubuntu-server:~$ nmap -sV -p 3306 10.74.2.69
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-14 08:45 UTC
Nmap scan report for ubuntu-server (10.74.2.69)
Host is up (0.000083s latency).

PORT     STATE SERVICE VERSION
3306/tcp open  mysql   MariaDB (unauthorized)

Service detection performed. Please report any incorrect results at https://nmap.o
rg/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.27 seconds
groupe-imad@ubuntu-server:~$ sudo sed -i 's/^bind-address.*/bind-address = 127.0.0
.1/' /etc/mysql/mariadb.conf.d/50-server.cnf
groupe-imad@ubuntu-server:~$ sudo systemctl restart mariadb
groupe-imad@ubuntu-server:~$ nmap -sV -p 3306 10.74.2.69
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-14 08:45 UTC
Nmap scan report for ubuntu-server (10.74.2.69)
Host is up (0.000085s latency).

PORT     STATE  SERVICE VERSION
3306/tcp closed mysql

Service detection performed. Please report any incorrect results at https://nmap.o
rg/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.16 seconds
groupe-imad@ubuntu-server:~$ sudo ss -tlnp | grep 3306
LISTEN 0      80         127.0.0.1:3306      0.0.0.0:*    users:(("mariadbd",pid=8
060,fd=23))
```

Figure 7.5 – Démonstration de l'exposition du port 3306 : `bind-address = 0.0.0.0` → Nmap détecte `3306/tcp open mysql MariaDB` ; après rétablissement en `127.0.0.1` → `3306/tcp closed` et `ss -tlnp` confirme l'écoute locale uniquement

Un simple changement de configuration rend la base détectable et joignable depuis le réseau. Un attaquant peut alors tenter une authentification par force brute ou exploiter d'éventuelles CVE sur la version exposée.

7.5 Injection SQL dans un script Python

Une démonstration d'injection SQL a ensuite été réalisée à partir d'un script Python connecté à MariaDB, sur la table suivante :

```sql
CREATE TABLE clients (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    nom   VARCHAR(100),
    email VARCHAR(100)
);
```

```
6 INSERT INTO clients (nom, email) VALUES
7     ('Dupont',  'dupont@techsud.fr'),
8     ('Martin',  'martin@techsud.fr'),
9     ('Bernard', 'bernard@techsud.fr');
```

Listing 7.4 – Table de test et données initiales

Version vulnérable

La première version du script construit la requête SQL par concaténation directe de la saisie utilisateur :

```
1 cursor.execute("SELECT * FROM clients WHERE nom = '" + nom +
    "'")
```

Listing 7.5 – Version vulnérable – concaténation directe

Avec l'entrée `' OR '1'='1`, la clause `WHERE` devient toujours vraie et retourne l'intégralité des enregistrements. D'autres charges utiles permettent d'aller plus loin : `' UNION SELECT 1, user(), database() -` expose le nom de l'utilisateur courant et de la base active.

Version corrigée

La version sécurisée utilise une requête paramétrée :

```
1 cursor.execute("SELECT * FROM clients WHERE nom = %s", (nom,)
    )
```

Listing 7.6 – Version sécurisée – requête paramétrée

La valeur saisie est transmise comme donnée et non comme fragment de SQL. Le connecteur l'échappe systématiquement avant l'envoi.

```
groupe-imad@ubuntu-server:~$ sudo python3 sql_injection_demo.py
============================================================
  DÉMONSTRATION INJECTION SQL — AEGIS / TechSud
============================================================

--- CAS 1 : Recherche normale ---

[VULNERABLE] Requête exécutée :
  SELECT id, nom, email FROM clients WHERE nom = 'Dupont'
  → Résultat : id=1, nom=Dupont, email=dupont@techsud.fr

[SECURISE] Requête template :
  SELECT id, nom, email FROM clients WHERE nom = %s
  Paramètre transmis séparément : 'Dupont'
  → Résultat : id=1, nom=Dupont, email=dupont@techsud.fr

--- CAS 2 : Injection — dump toute la table ---
  Payload injecté : ' OR '1'='1

[VULNERABLE] Requête exécutée :
  SELECT id, nom, email FROM clients WHERE nom = '' OR '1'='1'
  → Résultat : id=1, nom=Dupont, email=dupont@techsud.fr
  → Résultat : id=2, nom=Martin, email=martin@techsud.fr
  → Résultat : id=3, nom=Bernard, email=bernard@techsud.fr

[SECURISE] Requête template :
  SELECT id, nom, email FROM clients WHERE nom = %s
  Paramètre transmis séparément : "' OR '1'='1"
  → Aucun résultat (injection neutralisée).

--- CAS 3 : Injection UNION — lecture d'autres données ---
  Payload injecté : ' UNION SELECT 1, user(), database() --

[VULNERABLE] Requête exécutée :
  SELECT id, nom, email FROM clients WHERE nom = '' UNION SELECT 1, user(), databas
e() -- '
  → Résultat : id=1, nom=appuser@localhost, email=techsud_db

[SECURISE] Requête template :
  SELECT id, nom, email FROM clients WHERE nom = %s
  Paramètre transmis séparément : "' UNION SELECT 1, user(), database() -- "
  → Aucun résultat (injection neutralisée).
```

Figure 7.6 – Démonstration de l'injection SQL (cas 1 à 3) : recherche normale, dump de table par `OR '1'='1` et injection `UNION SELECT` – version vulnérable vs version sécurisée avec requête paramétrée

```
--- CAS 4 : Injection avec fermeture de guillemet ---
  Payload injecté : Dupont' --

[VULNERABLE] Requête exécutée :
  SELECT id, nom, email FROM clients WHERE nom = 'Dupont' -- '
  → Résultat : id=1, nom=Dupont, email=dupont@techsud.fr

[SECURISE] Requête template :
  SELECT id, nom, email FROM clients WHERE nom = %s
  Paramètre transmis séparément : "Dupont' -- "
  → Aucun résultat (injection neutralisée).

============================================================
  CONCLUSION
============================================================

  Version vulnérable :
    - Le payload est concaténé directement dans la requête SQL.
    - L'attaquant contrôle la structure de la requête.
    - Il peut lire toute la base, contourner l'authentification,
      exfiltrer des données sensibles, voire exécuter des commandes
      système (via LOAD_FILE / INTO OUTFILE selon les droits).

  Version sécurisée (requêtes paramétrées) :
    - La requête SQL est compilée AVANT d'y injecter les données.
    - Le connecteur échappe les caractères spéciaux (', --, etc.).
    - Le payload est traité comme une simple chaîne de caractères,
      jamais comme du code SQL.
    - Aucun des payloads ci-dessus ne fonctionne.
```

Figure 7.7 – Cas 4 (fermeture de guillemet) et conclusion : la version sécurisée neutralise l'ensemble des payloads – aucune injection ne passe avec les requêtes paramétrées

7.6 Audit MariaDB intégré au script Python

Le script d'audit a été enrichi pour vérifier plusieurs points de sécurité liés à MariaDB. Le compte utilisé pour ces vérifications est en lecture seule et les credentials sont chargés depuis un fichier de configuration externe, absent du dépôt Git.

```
groupe-imad@ubuntu-server:~$ sudo python3 audit_mariadb.py

  AEGIS — Audit MariaDB
  14/04/2026 09:44

  RÉSULTATS :
  Port 3306 local       : OUVERT
  Port 3306 externe     : fermé (bien)
  ☑ root_sans_mot_de_passe            : [OK      ] Root a un mot de passe défini
  ☑ comptes_sans_mot_de_passe         : [OK      ] Comptes sans mot de passe : NULL
  ☑ base_test_presente                : [OK      ] Base 'test' absente — bien
  ☑ bind_address                      : [OK      ] MariaDB écoute sur 127.0.0.1
  ☑ comptes_wildcard_host             : [OK      ] Aucun compte avec hôte wildcard '%'

[+] Rapport exporté : audit_mariadb.json
groupe-imad@ubuntu-server:~$
```

Figure 7.8 – Exécution du script `audit_mariadb.py` – tous les contrôles passent : port 3306 externe fermé, root protégé, aucun compte sans mot de passe, base `test` absente, `bind-address` sur 127.0.0.1

Chapitre 8

Phase 2 : Application web et attaques

8.1 Installation du serveur web Apache et de PHP

Le serveur web a été déployé via le service Apache2. Au niveau du pare-feu, le port `80/tcp` a été ouvert volontairement :

```
sudo ufw allow 80/tcp
```

Justification : toute la semaine a consisté à fermer des ports inutiles. Ouvrir le port 80 ici n'est pas une contradiction : c'est l'application du même principe, mais dans l'autre sens. Un service web est un besoin métier légitime et identifié. La règle n'est pas de tout fermer, c'est d'autoriser uniquement ce qui répond à un usage concret.

```
groupe-imad@ubuntu-server:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-04-14 07:34:08 UTC; 2h 30min ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 957 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 1006 (apache2)
      Tasks: 7 (limit: 4601)
     Memory: 19.7M (peak: 20.1M)
        CPU: 475ms
     CGroup: /system.slice/apache2.service
             ├─1006 /usr/sbin/apache2 -k start
             ├─1026 /usr/sbin/apache2 -k start
             ├─1028 /usr/sbin/apache2 -k start
             ├─1029 /usr/sbin/apache2 -k start
             ├─1030 /usr/sbin/apache2 -k start
             ├─1031 /usr/sbin/apache2 -k start
             └─9506 /usr/sbin/apache2 -k start

avril 14 07:34:08 ubuntu-server systemd[1]: Starting apache2.service - The Apache HTTP Server...
avril 14 07:34:08 ubuntu-server apachectl[989]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'Serv
avril 14 07:34:08 ubuntu-server systemd[1]: Started apache2.service - The Apache HTTP Server.

groupe-imad@ubuntu-server:~$ echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
<?php phpinfo(); ?>
groupe-imad@ubuntu-server:~$
```

Figure 8.1 – Statut du service Apache2 (actif) et création du fichier `/var/www/html/info.php` pour vérification de PHP

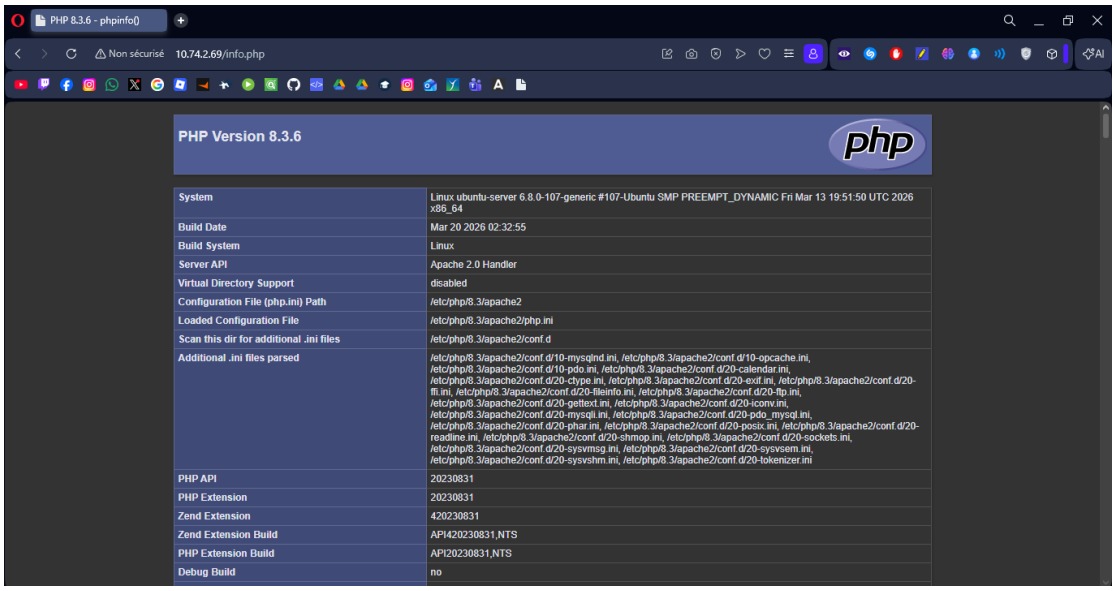


Figure 8.2 – Validation du bon fonctionnement de PHP 8.3.6 via le navigateur (page `phpinfo()` accessible sur 10.74.2.69)

8.2 Upload vulnérable et webshell

Un formulaire d'upload insuffisamment contrôlé permettait de déposer n'importe quel fichier dans `/var/www/html/upload/`. Un webshell minimal de la forme suivante a alors pu être téléversé :

```
<?php system($_GET['cmd']); ?>
```

Listing 8.1 – Webshell PHP minimal

Une fois le fichier envoyé, l'appel de type `/upload/shell.php?cmd=id` permettait d'exécuter une commande système via le serveur web, ce qui constitue une compromission critique.

```
groupe-imad@ubuntu-server:~$ cat > /tmp/shell.php << 'EOF'
<?php system($_GET['cmd']); ?>
EOF
groupe-imad@ubuntu-server:~$ curl -F "fichier=@/tmp/shell.php" http://localhost/
upload_vulnerable.php
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>TechSud — Formulaire de contact (VULNÉRABLE)</title>
    <style>
        body { font-family: sans-serif; max-width: 600px; margin: 60px auto; }
        .warning { background: #fff3cd; border: 1px solid #ffc107; padding: 10px
; margin-bottom: 20px; }
        .msg { background: #d4edda; padding: 10px; margin-top: 10px; }
    </style>
</head>
<body>
    <div class="warning">⚠ VERSION VULNÉRABLE — Usage pédagogique AEGIS uniqueme
nt</div>
    <h2>Envoyer un fichier</h2>
    <form method="POST" enctype="multipart/form-data">
        <input type="file" name="fichier" required><br><br>
        <button type="submit">Envoyer</button>
    </form>
            <div class="msg">Fichier uploadé : <a href='upload/shell.php'>shell.
php</a></div>
    </body>
</html>
groupe-imad@ubuntu-server:~$ curl "http://localhost/upload/shell.php?cmd=whoami"
www-data
groupe-imad@ubuntu-server:~$ curl "http://localhost/upload/shell.php?cmd=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)
groupe-imad@ubuntu-server:~$ curl "http://localhost/upload/shell.php?cmd=cat+/et
c/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
```

Figure 8.3 – Upload du webshell `shell.php` via `upload_vulnerable.php` et exécution de commandes système : `whoami` retourne `www-data`, `cat /etc/passwd` est accessible

8.3 Correction de la faille d'upload

Plusieurs contre-mesures ont ensuite été appliquées afin de neutraliser ce vecteur d'attaque :

1. vérification du type MIME avec `finfo_file()` côté serveur, indépendamment de l'extension déclarée par le client ;
2. refus de tout fichier dont le type MIME réel n'est pas une image reconnue (`image/jpeg`, `image/png`, `image/gif`, `image/webp`) ;
3. renommage systématique du fichier à l'arrivée avec un nom généré aléatoirement ;
4. ajout d'un fichier `.htaccess` dans le répertoire `upload/` pour désactiver l'interprétation PHP.

```
php_flag engine off
```

Listing 8.2 – Contenu du `.htaccess` placé dans `upload/`

```
groupe-imad@ubuntu-server:~$ curl "http://localhost/upload/shell.php?cmd=whoami" >
~/aegis-groupe-imad/tp_complementaire/phase2/webshell-demo-output.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100     9  100     9    0     0   2598      0 --:--:-- --:--:-- --:--:--  3000
groupe-imad@ubuntu-server:~$ curl -F "fichier=@/tmp/shell.php" http://localhost/up
load_securise.php
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>TechSud — Upload sécurisé</title>
    <style>
        body  { font-family: sans-serif; max-width: 600px; margin: 60px auto; }
        .ok   { background: #d4edda; border: 1px solid #28a745; padding: 10px; mar
gin-top: 10px; }
        .err  { background: #f8d7da; border: 1px solid #dc3545; padding: 10px; mar
gin-top: 10px; }
        .info { background: #d1ecf1; border: 1px solid #17a2b8; padding: 10px; mar
gin-bottom: 20px; font-size: 0.9em; }
    </style>
</head>
<body>
    <div class="info">
        ☑ Version sécurisée — MIME vérifié côté serveur, fichier renommé, PHP dés
activé dans /upload/
    </div>
    <h2>Envoyer une image</h2>
    <form method="POST" enctype="multipart/form-data">
        <input type="file" name="fichier" accept="image/*" required><br><br>
        <button type="submit">Envoyer</button>
    </form>
        <div class="err">Type de fichier refusé : text/x-php. Seules les images JP
EG/PNG/GIF/WebP sont acceptées.</div></body>
</html>
groupe-imad@ubuntu-server:~$ curl "http://localhost/upload/shell.php?cmd=whoami"
www-data
groupe-imad@ubuntu-server:~$
```

Figure 8.4 – Tentative d'upload du webshell sur `upload_securise.php` – le fichier est refusé (*Type de fichier refusé : text/x-php. Seules les images JPEG/PNG/GIF/WebP sont acceptées*) ; la requête `?cmd=whoami` retourne toujours `www-data` car le fichier n'a pas été stocké

8.4 En-têtes HTTP et réduction de l'empreinte du serveur

Les en-têtes HTTP et le niveau d'information renvoyé par Apache ont été corrigés afin d'éviter la divulgation de version et d'améliorer la protection navigateur. Les directives suivantes ont été ajoutées dans `/etc/apache2/conf-available/security.conf` :

```
ServerTokens Prod
ServerSignature Off
Header always set X-Frame-Options DENY
Header always set X-Content-Type-Options nosniff
```

```
Header always set X-XSS-Protection "1; mode=block"
```

Listing 8.3 – Directives de durcissement dans `security.conf`

Rôle de chaque directive :

- `ServerTokens Prod` et `ServerSignature Off` : suppriment la version d'Apache dans les en-têtes HTTP et les pages d'erreur.
- `X-Frame-Options DENY` : empêche le chargement de la page dans une *iframe*, bloquant les attaques de clicjacking.
- `X-Content-Type-Options nosniff` : interdit au navigateur d'interpréter un fichier dans un type MIME différent de celui déclaré.
- `X-XSS-Protection "1; mode=block"` : active le filtre XSS intégré des navigateurs qui le supportent.

```
groupe-imad@ubuntu-server:~$ sudo rm /var/www/html/info.php
groupe-imad@ubuntu-server:~$ sudo ufw allow 80/tcp comment "HTTP Apache"
Rule updated
Rule updated (v6)
groupe-imad@ubuntu-server:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
2222/tcp                   ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere                   # HTTP Apache
443/tcp                    ALLOW       Anywhere                   # HTTPS
2222/tcp (v6)              ALLOW       Anywhere (v6)
80/tcp (v6)                ALLOW       Anywhere (v6)              # HTTP Apache
443/tcp (v6)               ALLOW       Anywhere (v6)              # HTTPS

groupe-imad@ubuntu-server:~$ sudo cp upload_vulnerable.php /var/www/html/
groupe-imad@ubuntu-server:~$ sudo cp upload_securise.php   /var/www/html/
groupe-imad@ubuntu-server:~$ sudo cp xss_demo.php          /var/www/html/
groupe-imad@ubuntu-server:~$ sudo mkdir -p /var/www/html/upload
groupe-imad@ubuntu-server:~$ sudo chown www-data:www-data /var/www/html/upload
groupe-imad@ubuntu-server:~$ sudo chmod 755 /var/www/html/upload
groupe-imad@ubuntu-server:~$ sudo cp upload_htaccess.txt /var/www/html/upload/.hta
ccess
groupe-imad@ubuntu-server:~$ sudo a2enmod headers
Enabling module headers.
To activate the new configuration, you need to run:
  systemctl restart apache2
groupe-imad@ubuntu-server:~$ sudo systemctl restart apache2
groupe-imad@ubuntu-server:~$ sudo sed -i 's/^ServerTokens .*/ServerTokens Prod/' /
etc/apache2/conf-available/security.conf
groupe-imad@ubuntu-server:~$ sudo sed -i 's/^ServerSignature .*/ServerSignature Of
f/' /etc/apache2/conf-available/security.conf
groupe-imad@ubuntu-server:~$ sudo tee /etc/apache2/conf-available/security-headers
.conf << 'EOF'
Header always set X-Frame-Options DENY
Header always set X-Content-Type-Options nosniff
Header always set X-XSS-Protection "1; mode=block"
EOF
Header always set X-Frame-Options DENY
Header always set X-Content-Type-Options nosniff
Header always set X-XSS-Protection "1; mode=block"
groupe-imad@ubuntu-server:~$
```

Figure 8.5 – Configuration complète d'Apache : suppression de `info.php`, ouverture port 80 dans ufw, déploiement des scripts PHP, activation du module `headers` et injection des en-têtes de sécurité dans `security.conf`

```
groupe-imad@ubuntu-server:~$ sudo a2enconf security-headers
Enabling conf security-headers.
To activate the new configuration, you need to run:
  systemctl reload apache2
groupe-imad@ubuntu-server:~$ sudo a2enconf security
Conf security already enabled
groupe-imad@ubuntu-server:~$ sudo systemctl reload apache2
groupe-imad@ubuntu-server:~$ curl -I http://localhost
HTTP/1.1 200 OK
Date: Tue, 14 Apr 2026 10:13:10 GMT
Server: Apache
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Last-Modified: Mon, 13 Apr 2026 11:58:18 GMT
ETag: "29af-64f5632cabf7e"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

groupe-imad@ubuntu-server:~$
```

Figure 8.6 – Vérification des en-têtes HTTP avec `curl -I http://localhost` – `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` et `X-XSS-Protection` présents ; `Server: Apache` sans version

8.5 XSS réfléchie et correction

Une autre faille a été mise en évidence dans une fonctionnalité de recherche qui réinjectait directement la valeur du paramètre `q` dans la page.

```php
echo "Resultat pour : " . $_GET['q'];
```

Listing 8.4 – Affichage sans protection – vulnérable à la XSS

En appelant l'URL avec une charge utile de type `<script>alert('XSS')</script>`, le navigateur exécute le JavaScript injecté. La correction consiste à encoder la sortie HTML :

```php
echo "Resultat pour : " . htmlspecialchars($_GET['q'],
    ENT_QUOTES, 'UTF-8');
```

Listing 8.5 – Correction avec encodage HTML

Ce qu'un attaquant fait réellement avec une XSS : afficher une alerte est uniquement la preuve que le code s'exécute. En pratique, une XSS réfléchie est utilisée pour voler les cookies de session via `document.cookie`, rediriger la victime vers un site de phishing, afficher un faux formulaire de connexion, ou scanner le réseau interne depuis le navigateur de la victime. L'ensemble de ces attaques n'exige aucune interaction

de l'administrateur : il suffit que la victime clique sur un lien piégé contenant la charge utile dans l'URL.

Chapitre 9

Phase 3 : Logs et automatisation

9.1 Lecture manuelle des journaux

La phase de supervision a commencé par une lecture manuelle des principaux journaux système et applicatifs : `auth.log`, `syslog`, `fail2ban.log`, `access.log` et `error.log`. Cette étape permet de comprendre la structure des événements avant toute automatisation.

```
groupe-imad@ubuntu-server:~$ sudo tail -10 /var/log/auth.log
2026-04-14T10:17:01.840414+00:00 ubuntu-server CRON[9783]: pam_unix(cron:session):
 session opened for user root(uid=0) by root(uid=0)
2026-04-14T10:17:01.842957+00:00 ubuntu-server CRON[9783]: pam_unix(cron:session):
 session closed for user root
2026-04-14T10:25:01.095637+00:00 ubuntu-server CRON[9826]: pam_unix(cron:session):
 session opened for user root(uid=0) by root(uid=0)
2026-04-14T10:25:01.097292+00:00 ubuntu-server CRON[9826]: pam_unix(cron:session):
 session closed for user root
2026-04-14T10:25:09.098706+00:00 ubuntu-server sshd[9829]: Invalid user utilisateu
r_inexistant from 10.74.2.68 port 54221
2026-04-14T10:25:14.602092+00:00 ubuntu-server sshd[9829]: pam_unix(sshd:auth): ch
eck pass; user unknown
2026-04-14T10:25:14.602203+00:00 ubuntu-server sshd[9829]: pam_unix(sshd:auth): au
thentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.74.2.68
2026-04-14T10:25:16.191301+00:00 ubuntu-server sshd[9829]: Failed password for inv
alid user utilisateur_inexistant from 10.74.2.68 port 54221 ssh2
2026-04-14T10:25:25.138551+00:00 ubuntu-server sshd[9829]: Connection reset by inv
alid user utilisateur_inexistant 10.74.2.68 port 54221 [preauth]
2026-04-14T10:25:44.679703+00:00 ubuntu-server sudo: groupe-imad : TTY=pts/0 ; PWD
=/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/tail -10 /var/log/auth.log
groupe-imad@ubuntu-server:~$ sudo grep "Failed password" /var/log/auth.log | tail
-10
grep: /var/log/auth.log: binary file matches
2026-04-14T00:18:45.455618+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=
/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep 'Failed password' /var/log/a
uth.log
2026-04-14T00:19:30.601798+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=
/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep 'Failed password' /var/log/a
uth.log
groupe-imad@ubuntu-server:~$ sudo grep "Accepted" /var/log/auth.log | tail -5
grep: /var/log/auth.log: binary file matches
2026-04-13T12:55:10.785378+00:00 ubuntu-server sshd[1164]: Accepted password for g
roupe-imad from 10.74.2.22 port 61289 ssh2
2026-04-13T12:59:33.841599+00:00 ubuntu-server sshd[1256]: Accepted password for g
roupe-imad from 10.74.2.22 port 59078 ssh2
2026-04-14T00:20:07.622172+00:00 ubuntu-server sudo: groupe-imad : TTY=tty1 ; PWD=
/home/groupe-imad ; USER=root ; COMMAND=/usr/bin/grep Accepted /var/log/auth.log
2026-04-14T01:39:15.726605+00:00 ubuntu-server sshd[2918]: Accepted password for g
roupe-imad from 192.168.1.84 port 49175 ssh2
groupe-imad@ubuntu-server:~$ sudo grep "sudo" /var/log/auth.log | tail -5
grep: /var/log/auth.log: binary file matches
2026-04-14T01:38:29.642405+00:00 ubuntu-server sudo: pam_unix(sudo:session): sessi
```

Figure 9.1 – Lecture de `auth.log` : `tail -10` montrant des tentatives SSH sur utilisateur inexistant depuis 10.74.2.68, puis `grep` sur `Failed password`, `Invalid user` et `Accepted` pour isoler les événements clés

```
groupe-imad@ubuntu-server:~$ sudo tail -10 /var/log/syslog
2026-04-14T10:10:07.188850+00:00 ubuntu-server systemd[1]: Started apache2.service
 - The Apache HTTP Server.
2026-04-14T10:13:05.435028+00:00 ubuntu-server systemd[1]: Reloading apache2.servi
ce - The Apache HTTP Server...
2026-04-14T10:13:05.470775+00:00 ubuntu-server apachectl[9764]: AH00558: apache2:
Could not reliably determine the server's fully qualified domain name, using 127.0
.1.1. Set the 'ServerName' directive globally to suppress this message
2026-04-14T10:13:05.490578+00:00 ubuntu-server systemd[1]: Reloaded apache2.servic
e - The Apache HTTP Server.
2026-04-14T10:15:01.576634+00:00 ubuntu-server CRON[9774]: (root) CMD (command -v
debian-sal > /dev/null && debian-sal 1 1)
2026-04-14T10:17:01.840861+00:00 ubuntu-server CRON[9784]: (root) CMD (cd / && run
-parts --report /etc/cron.hourly)
2026-04-14T10:20:28.201794+00:00 ubuntu-server systemd[1]: Starting sysstat-collec
t.service - system activity accounting tool...
2026-04-14T10:20:28.204577+00:00 ubuntu-server systemd[1]: sysstat-collect.service
: Deactivated successfully.
2026-04-14T10:20:28.204866+00:00 ubuntu-server systemd[1]: Finished sysstat-collec
t.service - system activity accounting tool.
2026-04-14T10:25:01.096054+00:00 ubuntu-server CRON[9827]: (root) CMD (command -v
debian-sal > /dev/null && debian-sal 1 1)
groupe-imad@ubuntu-server:~$ sudo tail -10 /var/log/fail2ban.log
2026-04-14 07:34:08,701 fail2ban.filter         [796]: INFO      maxRetry: 3
2026-04-14 07:34:08,702 fail2ban.filter         [796]: INFO      findtime: 600
2026-04-14 07:34:08,702 fail2ban.actions        [796]: INFO      banTime: 86400
2026-04-14 07:34:08,702 fail2ban.filter         [796]: INFO      encoding: UTF-8
2026-04-14 07:34:08,703 fail2ban.filter         [796]: INFO    Added logfile: '/va
r/log/auth.log' (pos = 111386, hash = a7a8774ba266e58d9cbfc83eb318d90dae88280c)
2026-04-14 07:34:08,705 fail2ban.jail           [796]: INFO    Jail 'sshd' started
2026-04-14 08:05:01,777 fail2ban.filter         [796]: WARNING [sshd] Detected a l
og entry 2m 24s before the current time in operation mode. This looks like a laten
cy problem. Treating such entries as if they just happened.
2026-04-14 08:05:01,777 fail2ban.filter         [796]: WARNING [sshd] Please check
 a jail for a timing issue. Line with odd timestamp: 2026-04-14T08:02:37.951091+00
:00 ubuntu-server sudo: pam_unix(sudo:session): session closed for user root
2026-04-14 10:25:09,478 fail2ban.filter         [796]: INFO    [sshd] Found 10.74.
2.68 - 2026-04-14 10:25:09
2026-04-14 10:25:16,344 fail2ban.filter         [796]: INFO    [sshd] Found 10.74.
2.68 - 2026-04-14 10:25:16
```

Figure 9.2 – Consultation de `syslog` (démarrages Apache, jobs cron) et de `fail2ban.log` (configuration jail sshd, détection de timestamps anormaux)

```
groupe-imad@ubuntu-server:~$ sudo tail -5 /var/log/apache2/access.log
::1 - - [14/Apr/2026:10:19:03 +0000] "GET /upload/shell.php?cmd=id HTTP/1.1" 200 2
75 "-" "curl/8.5.0"
::1 - - [14/Apr/2026:10:19:16 +0000] "GET /upload/shell.php?cmd=cat+/etc/passwd HT
TP/1.1" 200 2208 "-" "curl/8.5.0"
::1 - - [14/Apr/2026:10:21:16 +0000] "GET /upload/shell.php?cmd=whoami HTTP/1.1" 2
00 229 "-" "curl/8.5.0"
::1 - - [14/Apr/2026:10:21:41 +0000] "POST /upload_securise.php HTTP/1.1" 200 1308
 "-" "curl/8.5.0"
::1 - - [14/Apr/2026:10:21:55 +0000] "GET /upload/shell.php?cmd=whoami HTTP/1.1" 2
00 229 "-" "curl/8.5.0"
groupe-imad@ubuntu-server:~$ sudo tail -5 /var/log/apache2/error.log
[Tue Apr 14 10:13:05.470965 2026] [mpm_prefork:notice] [pid 9697] AH00171: Gracefu
l restart requested, doing restart
AH00558: apache2: Could not reliably determine the server's fully qualified domain
 name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this m
essage
[Tue Apr 14 10:13:05.500467 2026] [mpm_prefork:notice] [pid 9697] AH00163: Apache/
2.4.58 (Ubuntu) configured -- resuming normal operations
[Tue Apr 14 10:13:05.500495 2026] [core:notice] [pid 9697] AH00094: Command line:
'/usr/sbin/apache2'
[Tue Apr 14 10:16:41.487336 2026] [php:error] [pid 9767] [client ::1:39134] script
 '/var/www/html/upload/shell.php' not found or unable to stat
groupe-imad@ubuntu-server:~$
```

Figure 9.3 – Consultation de `access.log` (requêtes vers `upload/shell.php?cmd=...`) et `error.log` Apache (erreur PHP sur fichier shell.php non trouvé après correction)

La ligne d'exemple suivante, observée dans `auth.log` après une tentative SSH échouée, illustre la structure typique d'un événement :

```
2026-04-14T10:25:09.098706+00:00 ubuntu-server sshd[9829]:
    Invalid user utilisateur_inexistant from 10.74.2.68 port
       54221
```

Listing 9.1 – Exemple de ligne d'échec SSH dans `auth.log`

Explication champ par champ :

- `2026-04-14T10:25:09...` : horodatage ISO 8601 avec fuseau horaire, format utilisé par `systemd-journald` sur Ubuntu 24.04.
- `ubuntu-server` : nom d'hôte de la machine qui a généré l'événement.
- `sshd[9829]` : nom du processus (`sshd`) suivi de son PID. Le PID permet de relier plusieurs lignes appartenant à la même session SSH.
- `Invalid user` : nature de l'événement. Indique que ce compte n'existe pas sur le système, caractéristique d'un balayage automatisé.
- `from 10.74.2.68` : adresse IP source du client. C'est ce champ qu'exploite fail2ban pour identifier et bannir les adresses répétitives.
- `port 54221` : port source éphémère du client.

9.2 Script Python : analyse de auth.log

Un script Python a été développé pour parser `auth.log` avec un argument `-hours`. La sortie JSON comporte notamment :

- le nombre de tentatives échouées par IP sur les dernières heures ;
- la liste des IP dépassant un seuil de cinq échecs ;
- les connexions réussies par utilisateur ;
- les commandes `sudo` horodatées.

Le script gère deux formats de date rencontrés selon les versions de Debian/Ubuntu :

- **Format classique syslog :** `Apr 14 10:15:22` – présent sur Debian 11 et versions antérieures.
- **Format ISO 8601 :** `2026-04-14T10:15:22.123456+02:00` – utilisé par `systemd-journald` sur Ubuntu 24.04.

```
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo pytho
n3 analyse_auth.py --hours 24

◖ AEGIS — Analyse auth.log (24h)
  Fichier : /var/log/auth.log

  Échecs SSH totaux      : 1
  IPs distinctes         : 1
  IPs suspectes (>5)     : 0
  Connexions réussies    : 6
  Commandes sudo         : 261

[+] Rapport exporté : analyse_auth.json
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo pytho
n3 analyse_auth.py --hours 48

◖ AEGIS — Analyse auth.log (48h)
  Fichier : /var/log/auth.log

  Échecs SSH totaux      : 1
  IPs distinctes         : 1
  IPs suspectes (>5)     : 0
  Connexions réussies    : 6
  Commandes sudo         : 262

[+] Rapport exporté : analyse_auth.json
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$
```

Figure 9.4 – Exécution du parseur `analyse_auth.py` avec `-hours 24` et `-hours 48` : 1 échec SSH, 1 IP distincte, 6 connexions réussies, 261 commandes sudo – export `analyse_auth.json`

9.3 Script Python : analyse de Apache access.log

Un second script a été consacré aux journaux d'accès du serveur web. Il détecte notamment :

1. les requêtes en erreur `4xx` et `5xx` ;
2. les motifs suspects dans l'URL : `/etc/passwd`, `/proc/`, `../`, `<script`, `SELECT`, `UNION`, `eval(` ;
3. le top 5 des *User-Agents* rares ou anormaux.

```
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo pytho
n3 analyse_apache.py

◖ AEGIS — Analyse Apache access.log
  Fichier : /var/log/apache2/access.log

  Requêtes totales      : 10
  Erreurs 4xx/5xx       : 5
  Requêtes suspectes    : 0
  IPs distinctes        : 2

[+] Rapport exporté : analyse_apache.json
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo chmod +x audit_auto.sh
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo bash audit_auto.sh

═══════════════════════════════════════════
  ◖ AEGIS — Audit automatique de sécurité
  14/04/2026 10:40:54
═══════════════════════════════════════════

[1] Pare-feu UFW
  [OK]   UFW est actif

[2] fail2ban
  [OK]   fail2ban actif — IPs bannies (sshd) : 0

[3] Port SSH
  [OK]   SSH écoute sur le port custom 2222 (pas sur 22)

[4] Accès root SSH
  [OK]   PermitRootLogin = no

[5] MariaDB bind-address
  [OK]   MariaDB écoute sur 127.0.0.1 uniquement

[6] Mises à jour de sécurité
  [OK]   Aucune mise à jour de sécurité en attente

[7] Ports en écoute inattendus
  [OK]   Aucun port inattendu détecté parmi les référentiels
  [INFO] Ports actifs : 2222 3306 53 80
```

Figure 9.5 – Exécution du parseur `analyse_apache.py` (10 requêtes totales, 5 erreurs 4xx/5xx, export `analyse_apache.json`), puis lancement de `audit_auto.sh` avec résultats colorés : UFW, fail2ban, SSH port 2222, PermitRootLogin, MariaDB, mises à jour – tous OK

9.4 Script Bash d'audit et exécution automatique par cron

Un script Bash d'audit système a été développé avec une sortie colorée (vert pour conforme, rouge pour non conforme). Il contrôle huit points essentiels :

1. état d'UFW ;
2. présence de fail2ban ;
3. configuration SSH (port différent de 22) ;
4. interdiction du compte root en SSH ;
5. écoute locale de MariaDB uniquement (`127.0.0.1`) ;
6. disponibilité des mises à jour de sécurité ;
7. ports suspects en écoute sur des numéros inattendus ;
8. fichiers modifiés dans `/etc/` dans les dernières 24 heures.

```
[8] Modifications récentes dans /etc/ (24h)
  [WARN] Fichiers modifiés dans /etc/ :
           /etc/ufw/after.rules
           /etc/ufw/user.rules
           /etc/ufw/user6.rules
           /etc/ufw/ufw.conf
           /etc/ufw/after6.rules
           /etc/default/ufw
           /etc/apache2/conf-available/security-headers.conf
           /etc/apache2/conf-available/security.conf
           /etc/fail2ban/jail.local
           /etc/mysql/debian.cnf
           /etc/mysql/mariadb.conf.d/50-server.cnf
           /etc/ssh/sshd_config
           /etc/ld.so.cache

═══════════════════════════════════════════════════
  ☑  Audit terminé — Aucun problème critique détecté
═══════════════════════════════════════════════════

groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo touch
 /var/log/audit_auto.log
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo chmod
 644 /var/log/audit_auto.log
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo bash
audit_auto.sh >> /var/log/audit_auto.log 2>&1
-bash: /var/log/audit_auto.log: Permission denied
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo chmod
 666 /var/log/audit_auto.log
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo bash
audit_auto.sh >> /var/log/audit_auto.log 2>&1
groupe-imad@ubuntu-server:~/aegis-groupe-imad/tp_complementaire/phase3$ sudo tail -50 /var/log/audit_auto.log
  14/04/2026 10:50:44
═══════════════════════════════════════════════════

[1] Pare-feu UFW
  [OK]   UFW est actif

[2] fail2ban
  [OK]   fail2ban actif — IPs bannies (sshd) : 0

[3] Port SSH
  [OK]   SSH écoute sur le port custom 2222 (pas sur 22)
```

Figure 9.6 – Suite de l'audit Bash : point [8] liste les fichiers modifiés dans `/etc/` (`ufw`, `apache2`, `fail2ban`, `mysql`, `sshd_config`) ; audit terminé sans problème critique ; test d'écriture dans `/var/log/audit_auto.log` et consultation via `tail -50`

Le script a ensuite été programmé dans la crontab afin de lancer l'audit toutes les heures et d'enregistrer le résultat dans un fichier dédié :

```
0 * * * * audit_auto.sh >> /var/log/audit_auto.log 2>&1
```

Listing 9.2 – Entrée crontab pour l'audit horaire automatisé

Chapitre 10

Cryptographie appliquée à TechSud

1. Pourquoi une clé SSH ed25519 vaut mieux qu'un mot de passe ?

Un mot de passe présente trois faiblesses structurelles : il peut être deviné par force brute si sa complexité est insuffisante, il peut être intercepté s'il transite d'une manière ou d'une autre en clair, et il peut être réutilisé sur plusieurs services, multipliant les risques en cas de fuite.

Une clé ed25519 fonctionne différemment. La sécurité repose sur la cryptographie sur courbes elliptiques (Curve25519). La clé privée ne quitte jamais la machine du collaborateur. Lors de la connexion, le serveur envoie un défi que seul le détenteur de la clé privée peut résoudre par signature cryptographique. Aucun secret n'est transmis sur le réseau, ce qui rend l'interception sans intérêt. ed25519 offre une sécurité équivalente à RSA 3072 bits avec des clés bien plus compactes et des opérations de signature plus rapides.

2. Le handshake SSH, étape par étape

Quand un collaborateur se connecte à la VM de TechSud via SSH, voici ce qui se passe avant que la session interactive s'ouvre :

1. **Échange de versions :** le client et le serveur s'annoncent mutuellement leur version du protocole SSH.
2. **Négociation des algorithmes :** les deux parties s'accordent sur les algorithmes à utiliser – chiffrement symétrique (AES), algorithme d'intégrité (SHA-2) et méthode d'échange de clés (ECDH).
3. **Établissement d'un secret partagé :** via ECDH, le client et le serveur calculent chacun de leur côté une valeur secrète identique, sans jamais se l'envoyer directement.
4. **Vérification de l'identité du serveur :** le serveur présente sa clé d'hôte. Le

client la compare à celle enregistrée dans `known_hosts`. Si elles diffèrent, c'est un signal d'alerte (*man-in-the-middle* possible).

5. **Authentification du client :** avec une clé ed25519, le serveur envoie un défi que le client signe avec sa clé privée. Le serveur vérifie la signature avec la clé publique enregistrée dans `authorized_keys`.
6. **Ouverture de la session :** si l'authentification réussit, la session interactive s'ouvre sur un canal entièrement chiffré.

3. HTTP versus HTTPS : ce qu'un attaquant capture avec Wireshark

Sur une connexion HTTP, Wireshark affiche tout en clair : l'URL complète, les paramètres des formulaires (identifiants, mots de passe), le contenu des pages et les cookies de session. Un attaquant positionné sur le même réseau peut lire ces données sans difficulté technique.

Sur HTTPS, Wireshark voit les paquets TCP mais leur contenu est chiffré. L'attaquant peut observer l'adresse IP du serveur et le nom de domaine dans le champ SNI, mais ni les URL précises, ni les données des formulaires, ni les cookies.

4. Certificat auto-signé versus certificat signé par une CA

Un certificat auto-signé est généré par TechSud elle-même. Le chiffrement fonctionne, mais les navigateurs affichent un avertissement car personne d'indépendant n'a vérifié l'identité du serveur.

Un certificat signé par une autorité de certification reconnue – comme Let's Encrypt, gratuit – garantit qu'une entité tierce a vérifié que le propriétaire du certificat contrôle réellement le nom de domaine.

Recommandation pour TechSud : déployer des certificats Let's Encrypt via Certbot sur toutes les interfaces exposées. Les renouvellements sont automatiques tous les 90 jours.

Conclusion générale

Ce projet AEGIS a couvert un cycle complet de sécurisation en deux phases complémentaires. Le TP principal a permis de construire un socle solide : déploiement d'une VM Ubuntu Server 24.04 LTS, durcissement SSH par clés ed25519, configuration du pare-feu ufw, mise en place de fail2ban et développement d'un premier script d'audit Python. Le TP complémentaire a approfondi trois domaines essentiels : sécurisation d'une base de données MariaDB (installation durcissante, moindre privilège, cloisonnement réseau, démonstration d'injection SQL), correction de vulnérabilités web (upload non contrôlé avec dépôt de webshell, XSS réfléchie, en-têtes HTTP) et automatisation de la supervision via l'analyse des journaux `auth.log` et Apache, complétée par un script Bash exécuté automatiquement en cron.

L'ensemble des livrables – scripts Python et Bash, fichiers PHP, sorties JSON et le présent rapport – est centralisé dans le dépôt GitHub du projet, dans le dossier `tp_complementaire/`.

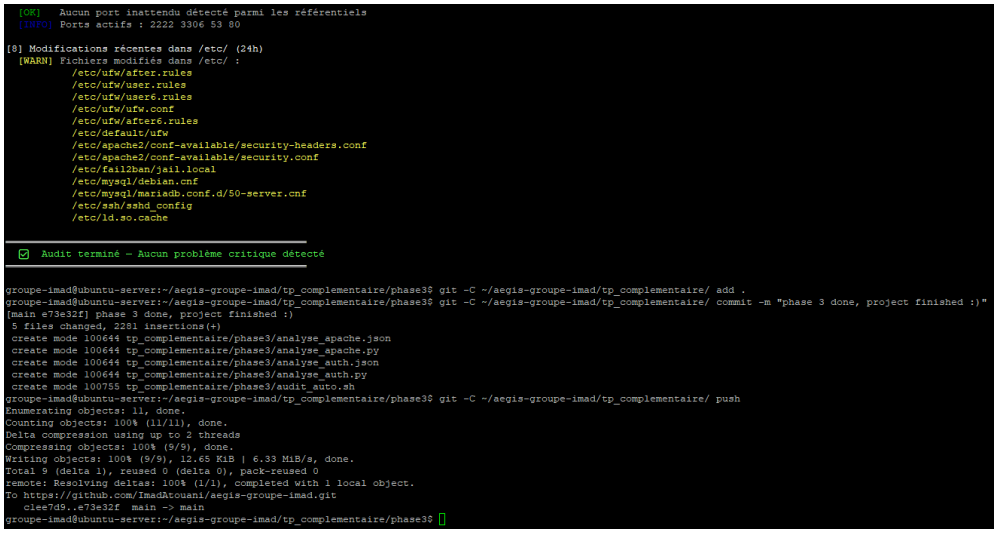


Figure 10.1 – Commit final du projet